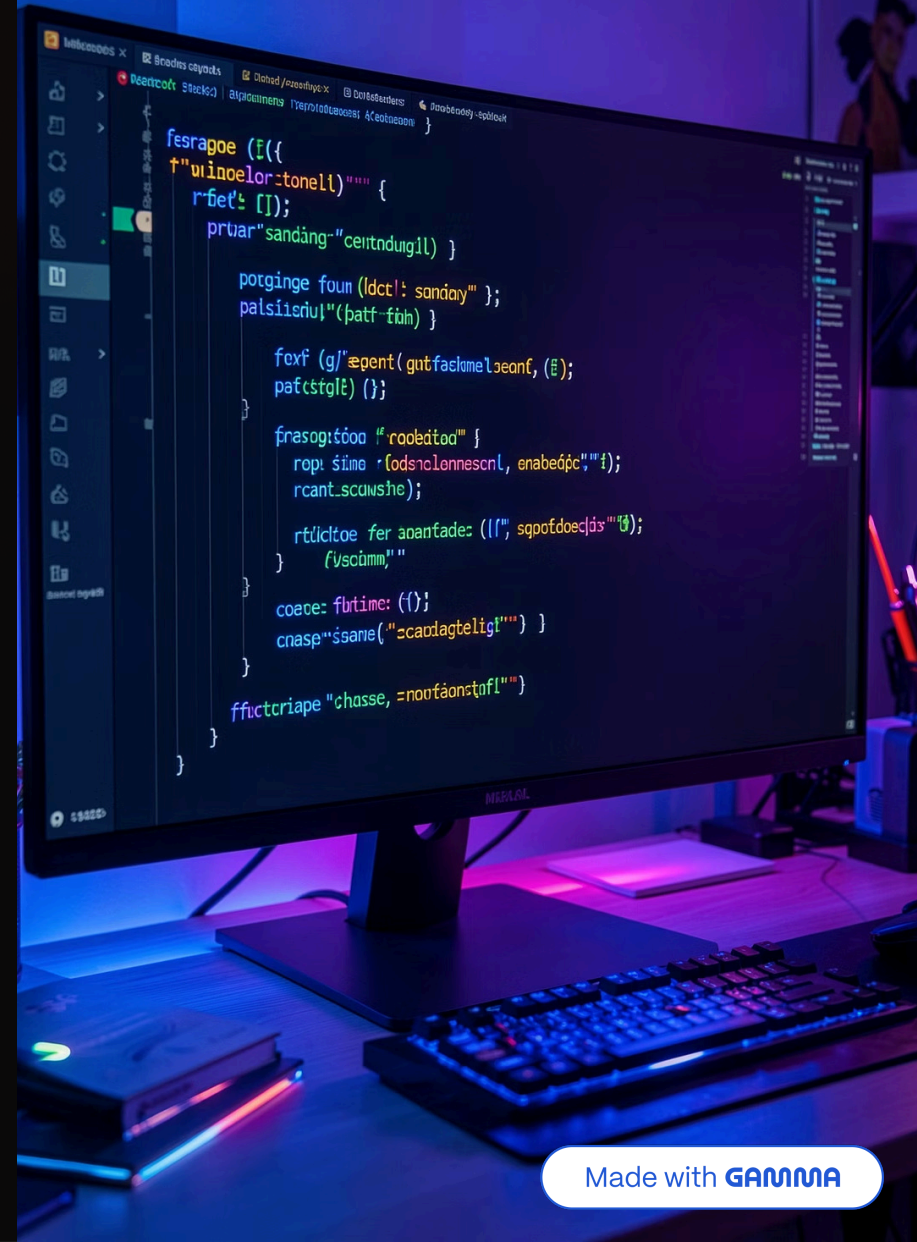


Introduction to Hoisting

Hoisting is JavaScript's behavior of moving declarations to the top of their scope before execution.

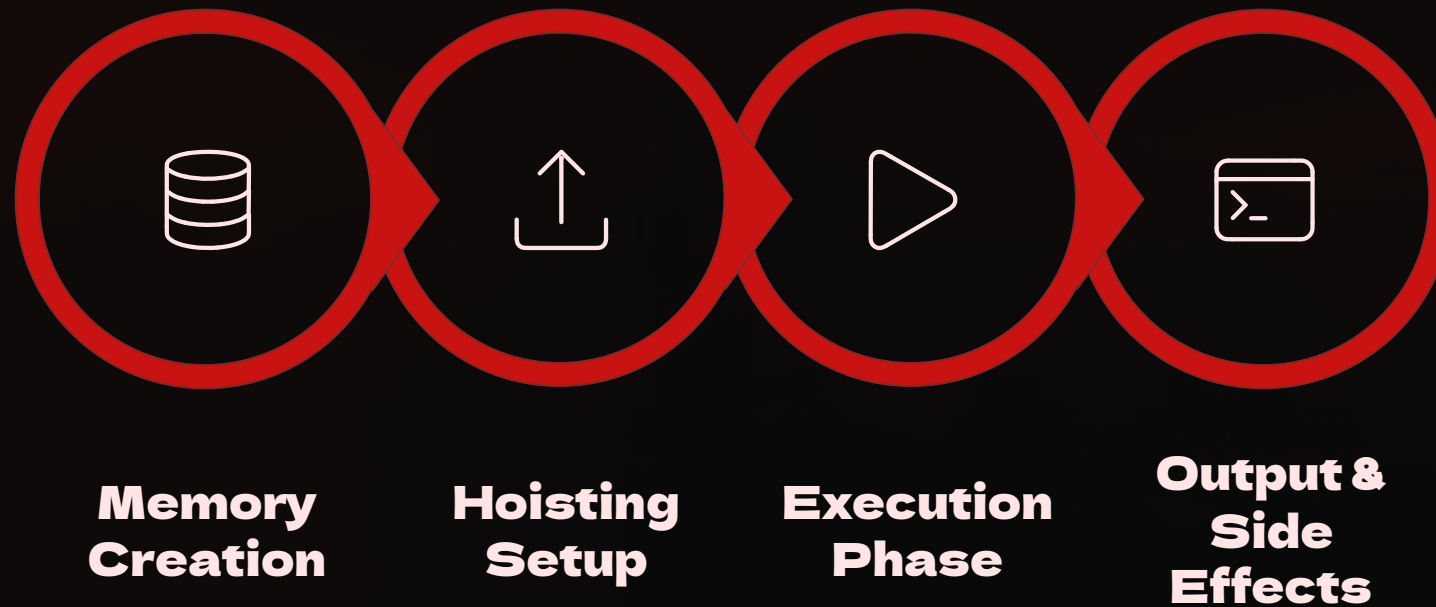
- ❏ **Important Point:** JavaScript does NOT move your code physically. Instead, the JavaScript Engine reads the entire code, allocates memory, then executes line by line.



JavaScript Execution Process

What Happens When JavaScript Runs?

JavaScript executes code in two phases.



JavaScript executes code in two distinct phases: Memory Creation and Execution.

Phase 1: Memory Creation

JavaScript scans code and allocates memory for:

- Variables
- Functions

Phase 2: Execution

JavaScript executes code line by line.

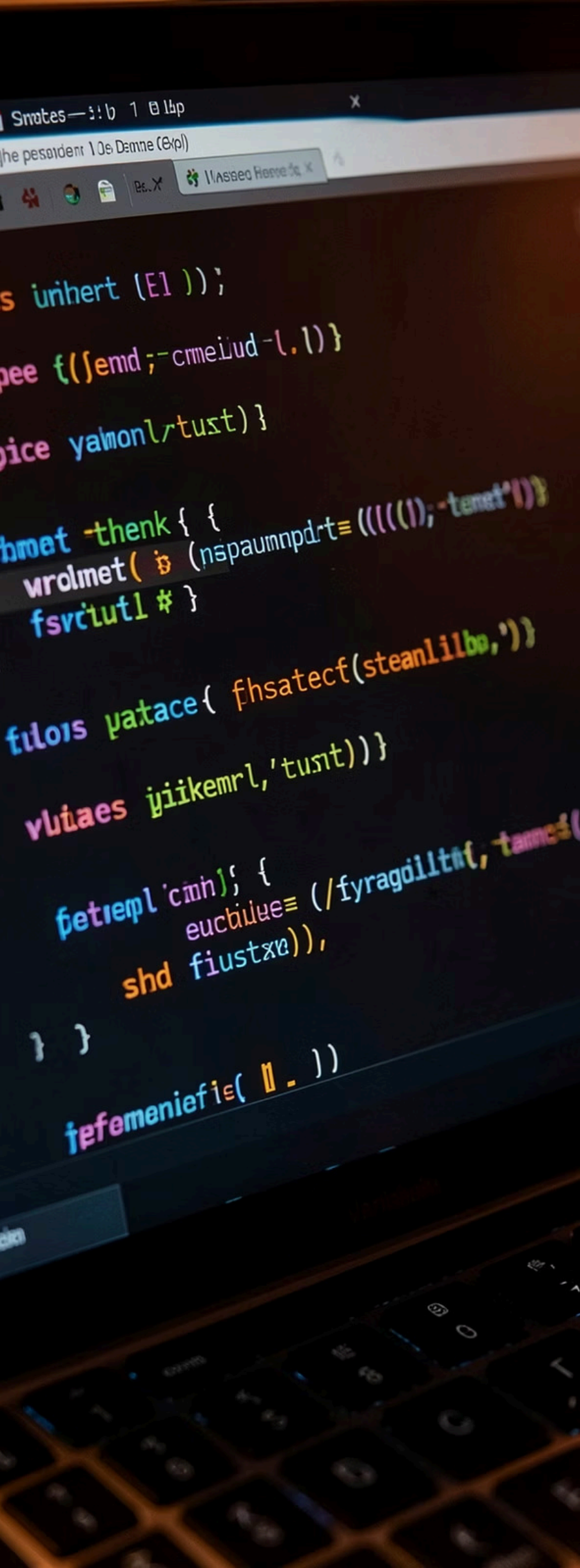
Behind the Scenes

Memory Phase: name → undefined

Execution Phase: name → Rahul

 **Example:** `var name = "Rahul"; console.log(name);` — Output: Rahul

 **Teaching Tip:** Draw two separate boxes — Memory Phase and Execution Phase. Students understand hoisting much faster.



Variable Hoisting with `var`

Variables declared using `var` are hoisted — JavaScript moves the declaration to the top, but not the assignment.

The Example

```
console.log(city);  
var city = "Hyderabad";
```

Output: undefined

Behind the Scenes

JavaScript internally rewrites it as:

```
var city;  
console.log(city);  
city = "Hyderabad";
```

Memory Phase

city → undefined

JavaScript knows the variable exists but hasn't assigned a value yet.

Execution Phase

city → Hyderabad

The assignment happens during execution, not memory creation.



Important Rule: `var` → Hoisted → Initialized as undefined. This is why we don't get an error — only undefined.



Teaching Tip: Ask students: "Why didn't we get an error?"

Hoisting with `let` and `const`

Unlike `var`, `let` and `const` are hoisted but **not initialized** — leading to a `ReferenceError` when accessed before declaration.

let Example

```
console.log(age);  
let age = 21;
```

Output: ReferenceError

const Example

```
console.log(pi);  
const pi = 3.14;
```

Output: ReferenceError

Visual Comparison

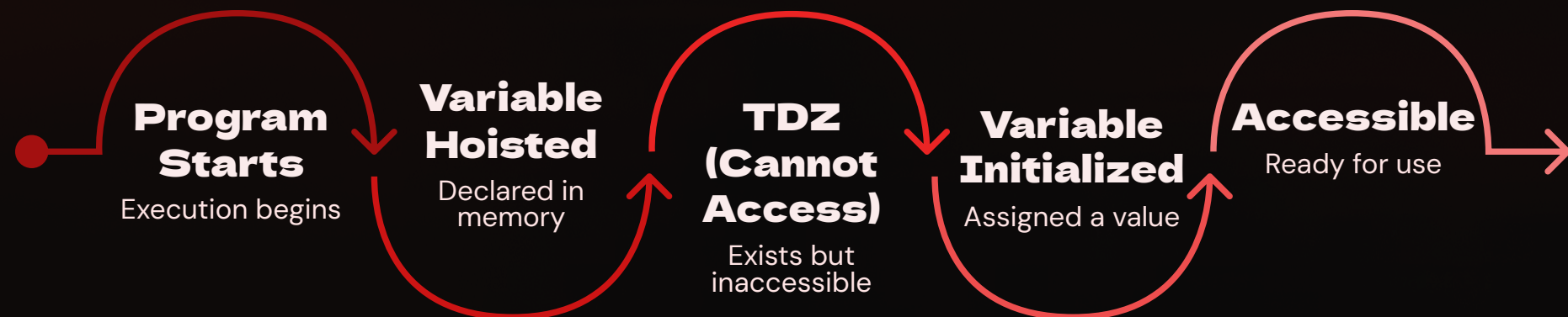
Keyword	Hoisted	Initial Value
var	Yes	undefined
let	Yes	No
const	Yes	No

📌 **Clarification:** Students usually think `let` and `const` are not hoisted. They *are* hoisted — but not initialized. That's the key difference.

Temporal Dead Zone (TDZ)

What is TDZ?

TDZ means: the period between variable hoisting and variable initialization. During this zone, the variable exists in memory but cannot be accessed.



The Temporal Dead Zone is the window where a variable has been hoisted but not yet initialized — accessing it throws a `ReferenceError`.

Example

```
console.log(name);  
let name = "Rahul";
```

Output: `ReferenceError`

Block Scope Example

```
{  
  console.log(age);  
  let age = 20;  
}
```

Output: `ReferenceError`

Why TDZ Exists

To prevent accidental access before initialization. It catches bugs early and enforces safer coding patterns.



Teaching Tip: Draw a timeline on the board showing the TDZ window. Students remember TDZ forever when they see it visually.

Function Hoisting

Function Declaration

Functions declared using the `function` keyword are **fully hoisted** — the entire function body is stored in memory during the creation phase.

The Example

```
greet();  
  
function greet(){  
  console.log("Hello");  
}
```

Output: Hello

Memory Phase

`greet` → **Entire Function**

JavaScript stores the complete function definition in memory before execution begins.

Execution Phase

`greet()` runs successfully — the function is fully available.



Important Rule: Function Declaration → Fully Hoisted. You can call a function before it's declared in the code.



Teaching Tip: Compare function hoisting with variable hoisting — functions are fully stored, while `var` only stores undefined.

Function Expression Hoisting

Function expressions behave differently from function declarations. Only the variable is hoisted — not the function assignment.

The Example

```
greet();  
var greet = function(){  
  console.log("Hello");  
}
```

Output: TypeError: greet is not a function

Why This Happens

Memory Phase: greet → undefined

Execution: greet(); becomes undefined();

Since greet holds undefined (not a function), calling it throws a TypeError.

Visual Comparison

Type	Hoisted
Function Declaration	Yes — fully
Function Expression	No — only variable



Very common interview question: Know the difference between function declarations and function expressions when it comes to hoisting.

JavaScript Execution Context

What is Execution Context?

Execution Context is the environment where JavaScript code executes. Think of it as a **workspace** created by JavaScript to run code.



Every execution context contains a Memory Component for storage and a Code Component for execution.



Memory Component

Stores all variables and functions declared in the current scope. Allocated during the Memory Creation Phase.



Code Component

Executes code line by line. Runs after memory has been allocated for all declarations.



Teaching Tip: Explain execution context as a workspace — JavaScript creates a dedicated space with memory and execution areas to run your code safely.

Hoisting Examples – Put It All Together

Run through these examples and predict the outputs before checking the results.

1

var Hoisting

```
console.log(a);  
var a = 10;
```

Output: undefined

2

let TDZ

```
console.log(a);  
let a = 10;
```

Output: ReferenceError

3

Function Declaration

```
show();  
function show(){  
  console.log("Hello");  
}
```

Output: Hello

4

Function Expression

```
show();  
var show = function(){  
  console.log("Hello");  
}
```

Output: TypeError



Mini Task: Students should run all four examples in the browser console and explain each output in their own words.

Homework & Learning Outcomes

Predict Output Exercises

Exercise 1

```
console.log(x);  
var x = 50;
```

Exercise 2

```
console.log(y);  
let y = 100;
```

Exercise 3

```
display();  
function display(){  
  
  console.log("JavaScript");  
}
```

Exercise 4

```
display();  
var display =  
  function(){  
    console.log("JS");  
  }
```

Final Outcome

After this lesson, students will:

- Understand Hoisting and how JavaScript moves declarations
- Understand Memory Creation Phase and Execution Phase
- Understand the Temporal Dead Zone (TDZ)
- Predict JavaScript output accurately
- Improve Debugging Skills



Teaching Tip: Hoisting is one of the most asked JavaScript interview topics. Make sure students can explain all four examples confidently.